

CSS Object-View-Box Property

Complete Guide: Image and Media Cropping

CSS Guide

May 10, 2026

Contents

1	Introduction	2
1.1	What is Object-View-Box?	2
1.2	Why Object-View-Box Matters	2
1.2.1	Real-World Applications	2
1.2.2	Benefits	3
1.3	Browser Support	3
2	Core Concepts	4
2.1	Understanding Object-View-Box Syntax	4
2.1.1	Inset Function	4
2.1.2	Values and Meanings	4
2.2	Combining with Object-Fit	4
3	Practical Examples	6
3.1	Example 1: Responsive Image Cropping	6
3.2	Example 2: Gallery Thumbnail Focus	6
3.3	Example 3: Aspect Ratio Adaptation	7
3.4	Example 4: SVG Pan and Zoom	8
3.5	Example 5: Video Thumbnail Extraction	8
4	Advanced Techniques	10
4.1	Dynamic Cropping with CSS Variables	10
4.1.1	Configurable Cropping	10
4.2	Combining with Transitions	10
4.3	Rounded Corners with Inset	11
5	Browser Support & Fallbacks	12
5.1	Current Support	12
5.2	Progressive Enhancement	12
6	Best Practices	13
6.1	Key Recommendations	13
6.1.1	1. Combine with Object-Fit	13
6.1.2	2. Test Different Screens	13
6.1.3	3. Use Variables for Consistency	13
6.1.4	4. Provide Fallback Imagery	14

7	Common Issues & Solutions	15
7.1	Issue 1: Crop Not Visible	15
7.2	Issue 2: Cropping Too Much	15
7.3	Issue 3: Not Working in Firefox	16
8	Summary & Quick Reference	17
8.1	When to Use Object-View-Box	17
8.2	Quick Reference	17
8.2.1	Common Patterns	17
8.2.2	Complete Example	18

Chapter 1

Introduction

1.1 What is Object-View-Box?

The `object-view-box` property defines a viewing window for replaced content (like images, videos, or SVG) similar to SVG's `viewBox`. It allows you to crop, zoom, or pan content within its container without modifying the source file or using JavaScript.

```
/* Basic syntax: crop and display portion of image */
object-view-box: inset(10% 20% 30% 40%);

/* Practical example: Focus on face in photo */
img {
  width: 200px;
  height: 200px;
  object-fit: cover;
  object-view-box: inset(20% 15%);
}
```

The `object-view-box` property is revolutionary for responsive image cropping, enabling content-aware resizing and dynamic focus regions without duplicating assets.

1.2 Why Object-View-Box Matters

1.2.1 Real-World Applications

- **Responsive Image Cropping:** Different crops for different screen sizes
- **Face Detection Focus:** Automatically focus on subjects in photos
- **Aspect Ratio Control:** Maintain subject focus while changing dimensions
- **Zoom Effects:** Magnify specific regions without loading different assets
- **SVG Panning:** Pan large SVG documents within small containers
- **Video Thumbnails:** Extract specific frame regions as previews
- **Grid Gallery Optimization:** Show different focal points across thumbnails

1.2.2 Benefits

- Single image asset, multiple visual presentations
- Reduces total file size (no duplicate crops)
- Improves responsive design workflows
- Better content control without source modification
- Pure CSS solution (no JavaScript needed)
- Works with any replaced element

1.3 Browser Support

Object-view-box support is emerging in modern browsers:

Browser	Version	Status
Chrome/Edge	v104+	Full Support
Firefox	Limited	Partial Support
Safari	v15.4+	Full Support
Opera	v90+	Full Support
IE 11	N/A	Not Supported

Chapter 2

Core Concepts

2.1 Understanding Object-View-Box Syntax

2.1.1 Inset Function

The primary syntax uses the `inset()` function to define cropping edges:

```
/* Single value: all sides */
object-view-box: inset(10%);

/* Two values: vertical, horizontal */
object-view-box: inset(10% 20%);

/* Three values: top, horizontal, bottom */
object-view-box: inset(10% 20% 30%);

/* Four values: top, right, bottom, left */
object-view-box: inset(10% 20% 30% 40%);

/* Mixed units */
object-view-box: inset(10px 20% 15px 25%);

/* Using round() for corners */
object-view-box: inset(10% round 5px);
```

2.1.2 Values and Meanings

Syntax	Meaning	Result
<code>inset(10%)</code>	Crop 10% from all sides	80% x 80% visible
<code>inset(10% 20%)</code>	Top/bottom 10%, left/right 20%	Cropped rectangle
<code>inset(10% 20% 30% 40%)</code>	Each side specified	Custom crop
<code>auto</code>	No cropping	Full image

2.2 Combining with Object-Fit

Object-view-box works in combination with `object-fit`:

```
/* Define viewing window, then how to fill it */
```

```
img {  
  width: 200px;  
  height: 200px;  
  
  /* Define what portion to view */  
  object-view-box: inset(20% 15%);  
  
  /* Define how to fill container */  
  object-fit: cover; /* or contain, fill, scale-down */  
}
```

Chapter 3

Practical Examples

3.1 Example 1: Responsive Image Cropping

```
/* Desktop: Show full image */
@media (min-width: 1024px) {
  .hero-image {
    width: 100%;
    height: 400px;
    object-fit: cover;
    object-view-box: auto; /* Full image */
  }
}

/* Tablet: Slight crop for aspect ratio */
@media (min-width: 768px) and (max-width: 1023px) {
  .hero-image {
    width: 100%;
    height: 300px;
    object-fit: cover;
    object-view-box: inset(10% 5%);
  }
}

/* Mobile: Focus on center face */
@media (max-width: 767px) {
  .hero-image {
    width: 100%;
    height: 200px;
    object-fit: cover;
    object-view-box: inset(15% 10%);
  }
}
```

3.2 Example 2: Gallery Thumbnail Focus

```
/* Gallery grid */
.gallery {
```

```
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
    gap: 10px;
}

.gallery-item {
    width: 100%;
    aspect-ratio: 1;
    object-fit: cover;
}

/* Each photo shows different focal point */
.gallery-item:nth-child(1) {
    object-view-box: inset(5% 10% 20% 15%);
}

.gallery-item:nth-child(2) {
    object-view-box: inset(20% 10% 10% 15%);
}

.gallery-item:nth-child(3) {
    object-view-box: inset(10% 20% 15% 5%);
}

.gallery-item:nth-child(4) {
    object-view-box: inset(15% 5% 20% 25%);
}

/* Hover: Show full image */
.gallery-item:hover {
    object-view-box: auto;
}
```

3.3 Example 3: Aspect Ratio Adaptation

```
/* Source image: 16:9 ratio */
.image-adaptive {
    object-fit: cover;
    object-view-box: auto;
}

/* Square aspect ratio: crop to square */
.image-square {
    width: 200px;
    height: 200px;
    object-view-box: inset(20%); /* Remove top/bottom */
}

/* Portrait aspect ratio: crop sides */
.image-portrait {
    width: 150px;
    height: 300px;
}
```

```
    object-view-box: inset(0% 30%); /* Remove left/right */
}

/* Ultra-wide: extract center strip */
.image-ultra-wide {
    width: 600px;
    height: 100px;
    object-view-box: inset(35% 0%); /* Show middle */
}
```

3.4 Example 4: SVG Pan and Zoom

```
/* Large SVG document in small container */
.svg-viewer {
    width: 300px;
    height: 300px;
    object-fit: none;
}

/* Pan to specific region */
.svg-viewer.region-1 {
    object-view-box: inset(0% 0% 50% 50%); /* Top-left */
}

.svg-viewer.region-2 {
    object-view-box: inset(0% 50% 50% 0%); /* Top-right */
}

.svg-viewer.region-3 {
    object-view-box: inset(50% 0% 0% 50%); /* Bottom-left */
}

.svg-viewer.region-4 {
    object-view-box: inset(50% 50% 0% 0%); /* Bottom-right */
}

/* Zoom: Smaller inset = zoom in */
.svg-viewer.zoomed {
    object-view-box: inset(25% 30% 25% 30%);
}

/* Full view: No inset */
.svg-viewer.full {
    object-view-box: auto;
}
```

3.5 Example 5: Video Thumbnail Extraction

```
/* Extract different frames from video */
video {
```

```
    width: 200px;
    height: 200px;
    object-fit: cover;
}

/* First video: Show 10 seconds into video */
#video-1 {
    object-view-box: inset(0% 25% 20% 15%);
}

/* Second video: Focus on action */
#video-2 {
    object-view-box: inset(15% 10% 15% 10%);
}

/* Third video: Show intro scene */
#video-3 {
    object-view-box: inset(20% 20% 30% 20%);
}

/* Hover to see full video */
video:hover {
    object-view-box: auto;
    height: 400px;
}
```

Chapter 4

Advanced Techniques

4.1 Dynamic Cropping with CSS Variables

4.1.1 Configurable Cropping

```
/* Define crop regions as variables */
:root {
  --crop-portrait: inset(10% 25%);
  --crop-square: inset(20%);
  --crop-landscape: inset(25% 10%);
  --crop-full: auto;
}

/* Use variables for different contexts */
.image {
  object-view-box: var(--crop-portrait);
}

.image.square {
  object-view-box: var(--crop-square);
}

.image.landscape {
  object-view-box: var(--crop-landscape);
}

/* Change crops based on context */
@media (max-width: 600px) {
  :root {
    --crop-portrait: inset(15% 20%);
    --crop-square: inset(25%);
  }
}
```

4.2 Combining with Transitions

```
/* Animate crop changes */
```

```
img {
  transition: object-view-box 0.3s ease;
  object-fit: cover;
  object-view-box: auto;
}

/* Crop on hover */
img:hover {
  object-view-box: inset(15% 10%);
}

/* Crop on focus */
img:focus {
  object-view-box: inset(10% 15%);
}

/* Smooth gallery transitions */
.gallery-item {
  transition: object-view-box 0.5s cubic-bezier(0.4, 0, 0.2, 1);
}

.gallery-item:nth-child(odd) {
  object-view-box: inset(10% 20%);
}

.gallery-item:nth-child(even) {
  object-view-box: inset(20% 10%);
}
```

4.3 Rounded Corners with Inset

```
/* Crop with rounded corners */
img {
  width: 200px;
  height: 200px;
  object-fit: cover;
  object-view-box: inset(10% round 10px);
  border-radius: 10px;
}

/* Different border radius per crop */
.image-soft {
  object-view-box: inset(15% round 5px);
}

.image-medium {
  object-view-box: inset(15% round 15px);
}

.image-harsh {
  object-view-box: inset(15%); /* No rounding */
}
```

Chapter 5

Browser Support & Fallbacks

5.1 Current Support

Feature	Chrome	Safari	Firefox
object-view-box	v104+	v15.4+	Limited
inset() function	v104+	v15.4+	Partial
Transitions	v104+	v15.4+	Limited

5.2 Progressive Enhancement

```
/* Base: Works everywhere */
img {
  width: 100%;
  height: auto;
}

/* Enhanced: For modern browsers */
@supports (object-view-box: inset(0%)) {
  img {
    object-view-box: inset(15% 10%);
  }
}

/* Fallback: Use width + height for cropping */
@supports not (object-view-box: inset(0%)) {
  img {
    width: 100%;
    height: 100%;
    object-fit: cover;
    /* Cropping not available */
  }
}
```

Chapter 6

Best Practices

6.1 Key Recommendations

6.1.1 1. Combine with Object-Fit

```
/* Good: Both properties work together */
img {
  object-view-box: inset(15% 10%);
  object-fit: cover;
}

/* Less effective: Without object-fit */
img {
  object-view-box: inset(15% 10%);
  /* How does it fill container? */
}
```

6.1.2 2. Test Different Screens

```
/* Desktop: minimal crop */
@media (min-width: 1024px) {
  .image { object-view-box: inset(5%); }
}

/* Tablet: moderate crop */
@media (max-width: 1023px) {
  .image { object-view-box: inset(10%); }
}

/* Mobile: aggressive crop to focus */
@media (max-width: 600px) {
  .image { object-view-box: inset(20%); }
}
```

6.1.3 3. Use Variables for Consistency

```
:root {
  --crop-standard: inset(10% 15%);
  --crop-tight: inset(15% 20%);
  --crop-loose: inset(5%);
}

.primary-image {
  object-view-box: var(--crop-standard);
}

.thumbnail {
  object-view-box: var(--crop-tight);
}

.background {
  object-view-box: var(--crop-loose);
}
```

6.1.4 4. Provide Fallback Imagery

```
/* If object-view-box not supported */
img {
  background-image: url('fallback-cropped.jpg');
  object-view-box: inset(15% 10%);
}

/* Or use different image */
@supports not (object-view-box: inset(0%)) {
  img[data-crop] {
    content: url('cropped-version.jpg');
  }
}
```

Chapter 7

Common Issues & Solutions

7.1 Issue 1: Crop Not Visible

Problem: object-view-box doesn't seem to crop anything.

Cause: Missing object-fit or incorrect values.

Solution:

```
/* Wrong: No object-fit */
img {
  object-view-box: inset(15%); /* Doesn't show effect */
}

/* Correct: Include object-fit */
img {
  object-fit: cover;
  object-view-box: inset(15%);
}

/* Ensure container has dimensions */
img {
  width: 200px;
  height: 200px;
  object-fit: cover;
  object-view-box: inset(15%);
}
```

7.2 Issue 2: Cropping Too Much

Problem: Content disappears or looks wrong.

Solution:

```
/* Start with small values */
img {
  object-view-box: inset(5% 10%); /* Small crop */
}

/* Increase if needed */
img:hover {
```

```
    object-view-box: inset(15% 20%); /* Larger crop */
}

/* Debug: Visualize crop area */
img {
    border: 3px solid red;
    object-view-box: inset(10%);
}
```

7.3 Issue 3: Not Working in Firefox

Problem: Firefox doesn't support object-view-box.

Solution:

```
/* Use @supports for feature detection */
@supports (object-view-box: inset(0%)) {
    img {
        object-view-box: inset(15%);
    }
}

/* Firefox fallback: Use CSS filter */
@supports not (object-view-box: inset(0%)) {
    img {
        clip-path: inset(15% 10% 15% 10%);
    }
}
```

Chapter 8

Summary & Quick Reference

8.1 When to Use Object-View-Box

Use `object-view-box` when:

- Need different crops for different devices
- Single asset, multiple presentations
- Focus on specific subjects in photos
- Extract regions from large images
- Pan SVG documents
- Create responsive galleries

8.2 Quick Reference

8.2.1 Common Patterns

```
/* No cropping */
object-view-box: auto;

/* Uniform crop from all sides */
object-view-box: inset(15%);

/* Horizontal + vertical crop */
object-view-box: inset(10% 20%);

/* Custom all sides */
object-view-box: inset(10% 15% 20% 25%);

/* Crop with rounded corners */
object-view-box: inset(10% round 5px);
```

8.2.2 Complete Example

```
img {  
  width: 100%;  
  height: auto;  
  object-fit: cover;  
  transition: object-view-box 0.3s ease;  
  object-view-box: auto;  
}  
  
img:hover {  
  object-view-box: inset(15% 10%);  
}  
  
@media (max-width: 600px) {  
  img {  
    object-view-box: inset(20% 15%);  
  }  
}
```

Thank you for reading!

For more information, visit [MDN Web Docs](#) and [web.dev](#)